

使用 ADK 建構多代理系統

來源：<https://codelabs.developers.google.com/codelabs/production-ready-ai-with-gc/3-developing-agents/build-a-multi-agent-system-with-adk?hl=zh-tw>

步驟數：14

「瞭解如何使用 Google 代理開發套件 (ADK) 協調多代理系統。」

目錄

1. 簡介
2. 多代理系統
3. 專案設定
4. 開啟 Cloud Shell 編輯器
5. 啟用 API
6. 設定專案環境
7. 瞭解父代理、子代理和同層代理間的轉移方向
8. 使用工作階段狀態來儲存及擷取資訊
9. 工作流程代理
10. 使用 SequentialAgent 建構多代理系統
11. 新增 LoopAgent 來執行疊代作業
12. 使用 ParallelAgent 執行「擴散收集」作業
13. 自訂工作流程代理
14. 恭喜！

步驟 1 · 約 0 分鐘

1. 簡介

總覽

本實驗室說明如何使用 Google [Agent Development Kit \(Google ADK\)](#)，自動調度管理複雜的多代理系統。您將從簡單的代理程式階層，進展到建構自動化協作工作流程。

建構項目

您將建構兩個不同的多代理系統：

- 簡單的**旅遊規劃代理**，可學習在「腦力激盪」代理和「景點規劃」代理之間轉移對話。
- 更進階的**電影提案生成器**，會使用自動化代理程式 (例如研究人員、編劇和影評) 組成的「編劇室」，以迴圈形式共同作業，生成完整電影情節。

課程內容

- 如何建立上層和子代理關係。
 - 如何從工具將資料寫入工作階段 `state`。
 - 如何使用索引鍵範本 (例如 `{my_key?}`) 從 `state` 讀取資料。
 - 如何使用 `SequentialAgent` 建立逐步工作流程。
 - 如何使用 `LoopAgent` 建立疊代精進週期。
 - 如何使用 `ParallelAgent` 並行執行獨立工作。
-

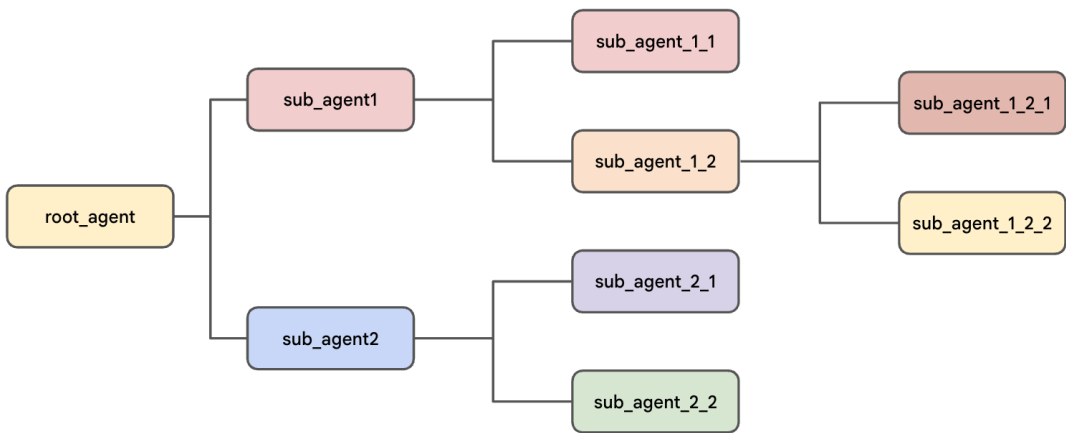
2. 多代理系統

開發人員可以使用 [Agent Development Kit \(ADK\)](#)，讓生成式模型的多步驟行為更可靠精細。您不必編寫複雜的提示詞，只要使用 ADK 建構多個簡單的代理流程，再劃分工作，代理就會合作解決問題。

與使用單一的整體式提示相比，這種方法有幾個優點：

- **設計更簡單**：設計及安排小型專業代理程式流程，比設計大型複雜提示更簡單。
- **可靠性**：相較於大型複雜的代理，專用代理執行特定工作時更可靠。
- **可維護性**：修正或改良小型專用代理時，不會影響系統其他部分。
- **模組化**：為某個工作流程建構的代理，可輕鬆重複用於其他工作流程。

階層式代理樹狀結構



在 ADK 中，您可以將代理整理成**樹狀結構**，這個層級架構是控管對話流程的關鍵，因為它會限制哪些服務專員可以將對話「轉移」給其他服務專員。這樣一來，系統行為會更可預測，也更容易偵錯。好處包括：

- **直覺式設計**：結構靈感來自現實世界的團隊，更容易推論。
- **受控流程**：階層可讓您精確控制工作委派，有助於偵錯。舉例來說，即使您有兩個描述類似的代理程式，樹狀結構也能確保呼叫**正確**的報告撰寫代理程式。

整個結構以 **root_agent** 開頭。這個代理是**父代理**，可有一或多個**子代理**，而子代理本身也可以是父代理，底下有自己的子代理，形成樹狀結構。

步驟 3 · 約 0 分鐘

3. 專案設定

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

info

從做中學 免付費

立即取得 Google Cloud 抵免額，並完成這個實驗室。無須提供信用卡或付款方式。

啟用

Google 帳戶

如果沒有個人 Google 帳戶，請[建立 Google 帳戶](#)。

請使用個人帳戶，而非公司或學校帳戶。

公司和學校帳戶可能設有限制，導致您無法啟用本實驗室所需的 API。

登入 Google Cloud 控制台

使用個人 Google 帳戶登入 [Google Cloud 控制台](#)。

啟用計費功能

兌換 Google Cloud 抵免額 (選用)

如要參加這個研討會，您需要有具備抵免額的帳單帳戶。請使用本程式碼研究室頂端橫幅中的抵免額開始操作。如果已連結帳單帳戶，可以略過這個步驟。

設定個人帳單帳戶

如果使用 Google Cloud 抵免額設定計費，可以略過這個步驟。

如要設定個人帳單帳戶，請[前往這裡在 Cloud 控制台中啟用帳單](#)。

注意事項：

- 完成本實驗室的 Cloud 資源費用應不到 \$1 美元。
- 您可以按照本實驗室結尾的步驟刪除資源，以免產生後續費用。
- 新使用者可獲得價值 **\$300 美元**的免費試用期。

建立專案 (選用)

如果沒有要用於本實驗室的現有專案，請[在這裡建立新專案](#)。

步驟 4 · 約 0 分鐘

4. 開啟 Cloud Shell 編輯器

1. 按一下這個連結，直接前往 [Cloud Shell 編輯器](#)
2. 如果系統在今天任何時間提示您授權，請點選「授權」繼續操作。

Authorize Cloud Shell

gcloud is requesting your credentials to make a GCP API call.

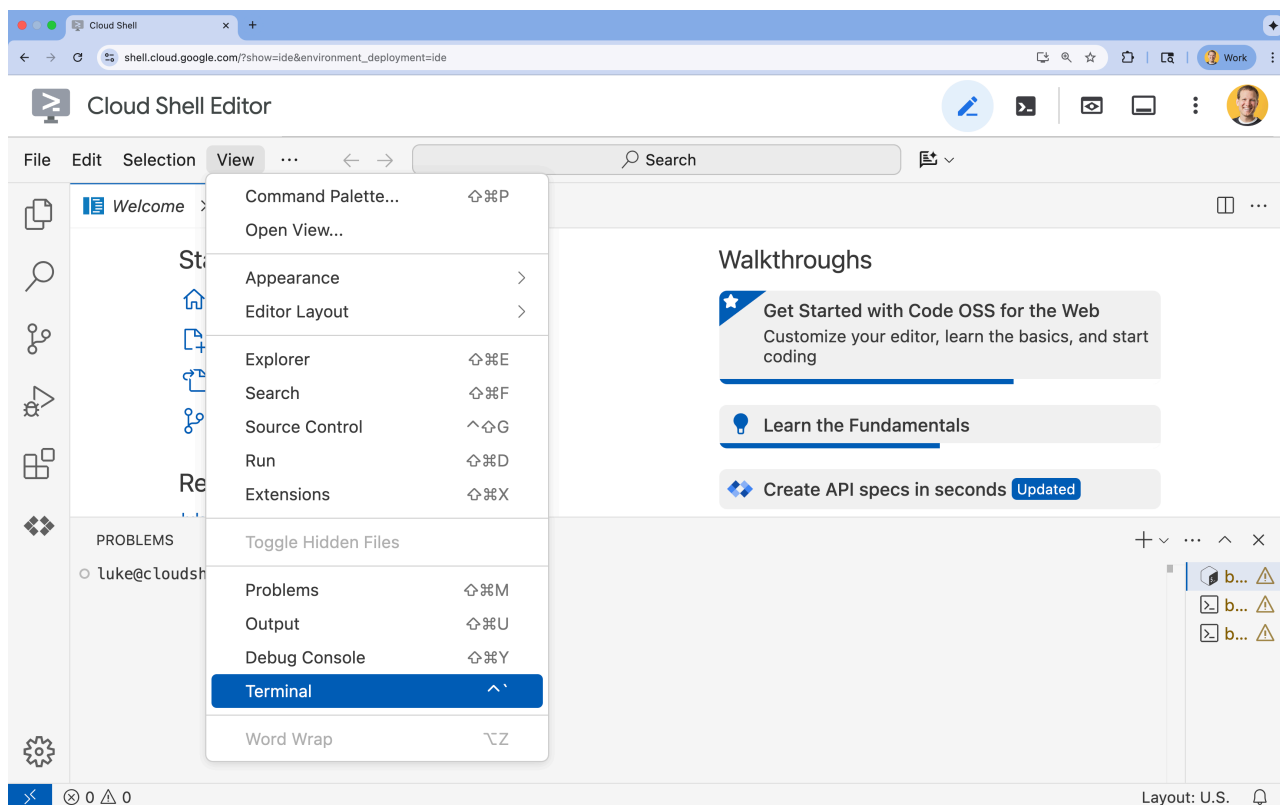
Click to authorize this and future calls that require your credentials.

REJECT

AUTHORIZE

3. 如果畫面底部未顯示終端機，請開啟終端機：

- 按一下「查看」
- 按一下「終端機」



4. 在終端機中，使用下列指令設定專案：

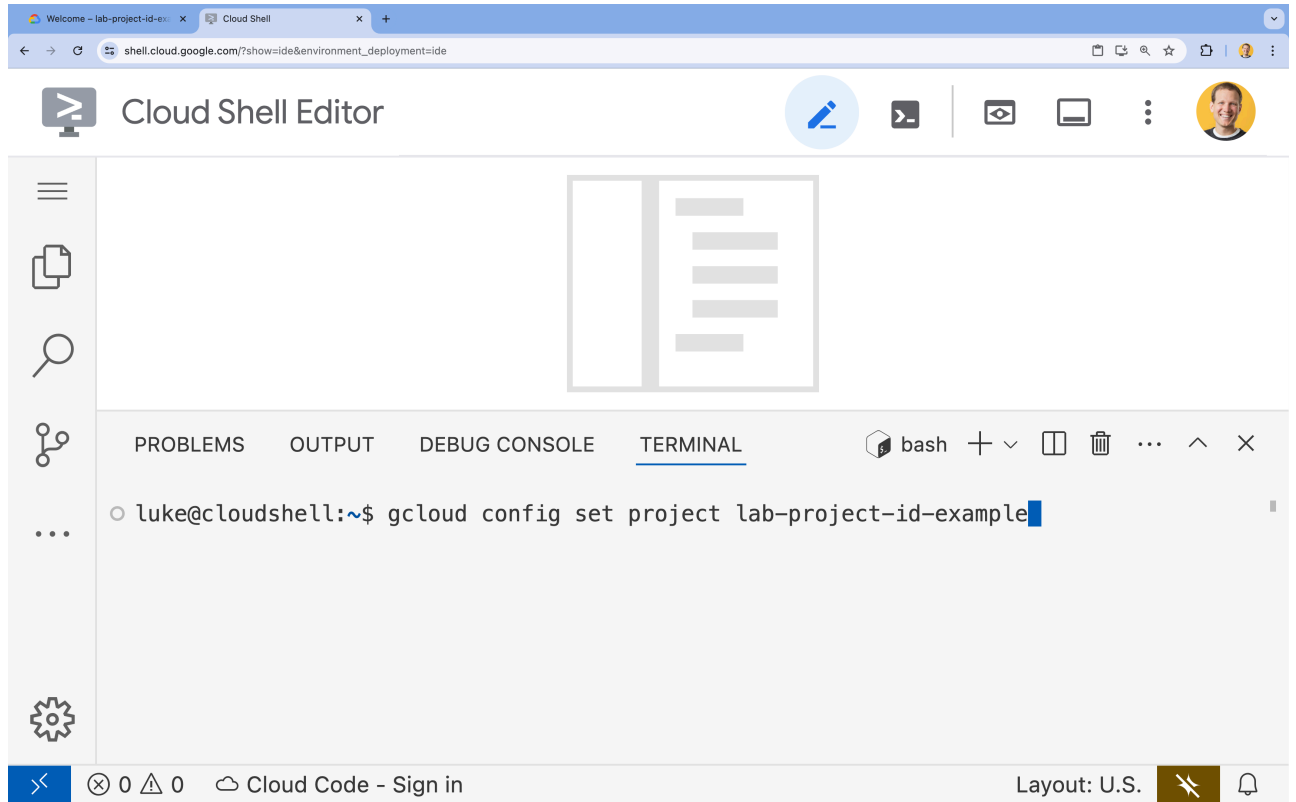
```
gcloud config set project [PROJECT_ID]
```

- 範例：

```
gcloud config set project lab-project-id-example
```

- 如果忘記專案 ID，可以使用下列指令列出所有專案 ID：

```
gcloud projects list
```



5. 您應會看到下列訊息：

```
Updated property [core/project].
```

如果看到 **WARNING** 並收到 **Do you want to continue (Y/n)?** 提示，表示您可能輸入了錯誤的專案 ID。按下 **n** 和 **Enter**，然後再次嘗試執行 **gcloud config set project** 指令。

步驟 5 · 約 0 分鐘

5. 啟用 API

如要使用 [Vertex AI API](#) 並與 Gemini 模型互動，您需要在 Google Cloud 雲端專案中啟用 Vertex AI API。

1. 在終端機中啟用 API：

```
gcloud services enable aiplatform.googleapis.com
```

Python 適用的 Vertex AI SDK 簡介

如要從 Python 應用程式與 Vertex AI 上代管的模型互動，請使用 Vertex AI SDK for Python。這個 SDK 可簡化傳送提示、指定模型參數及接收回覆的程序，不必直接處理基礎 API 呼叫的複雜性。

如需 Python 適用的 Vertex AI SDK 完整說明文件，請參閱「[Python 適用的 Vertex AI SDK 簡介 | Google Cloud](#)」。

6. 設定專案環境

建立存放區的本機複本

1. 在終端機中，複製含有範例檔案的存放區。

```
git clone --depth 1 https://github.com/GoogleCloudPlatform/devrel-demos.git devrel-demos-multiagent-lab
```

`--depth 1` 旗標只會複製最新版本，速度較快。

2. 在終端機中，將實驗室專屬資料夾移至主目錄，並重新命名，使其符合實驗室的預期結構。

```
mv devrel-demos-multiagent-lab/ai-ml/build-multiagent-systems-with-adk/adk_multiagent_systems ~
```

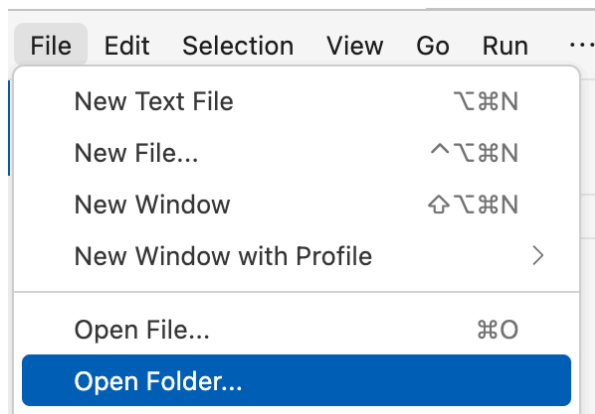
3. 在終端機中，前往本實驗室的正確工作目錄 (`adk_multiagent_systems`)。

```
cd ~/adk_multiagent_systems
```

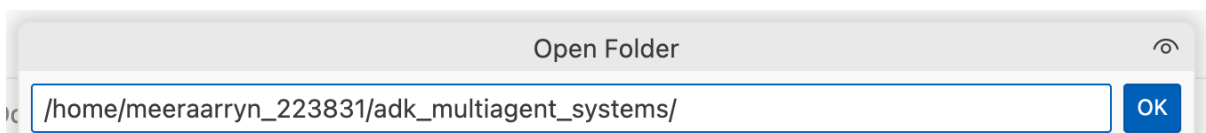
檢查檔案結構

現在所有檔案都已建立，請在檔案總管中開啟 `adk_multiagent_systems` 資料夾，查看完整結構。

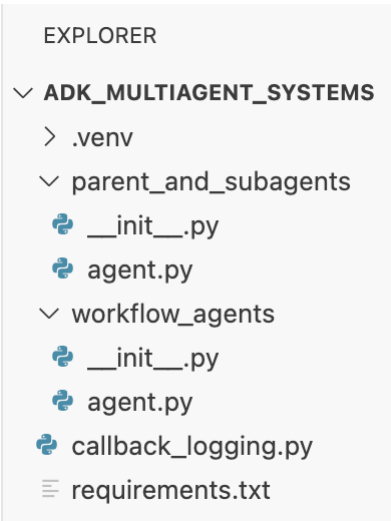
1. 在 Cloud Shell 編輯器選單中，依序選取「File」>「Open Folder...」。



2. 在彈出式方塊中，於使用者名稱後方新增下列資料夾資訊：`adk_multiagent_systems/`。按一下「確定」。應該會類似這樣：



3. 左側的「探索」面板會重新整理。您現在應該會看到完整的專案結構，其中包含 `parent_and_subagents` 和 `workflow_agents` 子目錄，可供進行後續步驟。



如果終端機因此消失，請依序點選頂端選單中的「View」(檢視)和「Terminal」(終端機)，重新開啟終端機。

啟動虛擬環境

1. 在終端機中，使用 `uv` 建立並啟動虛擬環境。確保專案依附元件不會與系統 Python 或其他專案發生衝突。

```
uv venv
source .venv/bin/activate
```

如果 **Cloud Shell** 工作階段重新整理或開啟新的終端機分頁，您可能需要從 `adk_multiagent_systems` 目錄中執行 `source .venv/bin/activate`，重新啟動虛擬環境。

2. 在終端機中，從 `requirements.txt` 檔案安裝 `google-adk` 和其他依附元件：

```
uv pip install -r requirements.txt
```

設定環境變數

1. 你已在 `adk_multiagent_systems` 目錄中。在終端機中，建立 `.env` 檔案來儲存環境變數：

```
cloudshell edit .env
```

2. 將下列內容貼到編輯器中開啟的 `.env` 檔案：

```
GOOGLE_GENAI_USE_VERTEXAI=TRUE
GOOGLE_CLOUD_PROJECT="[YOUR-PROJECT-ID]"
GOOGLE_CLOUD_LOCATION=global
MODEL="gemini-2.5-flash"
```

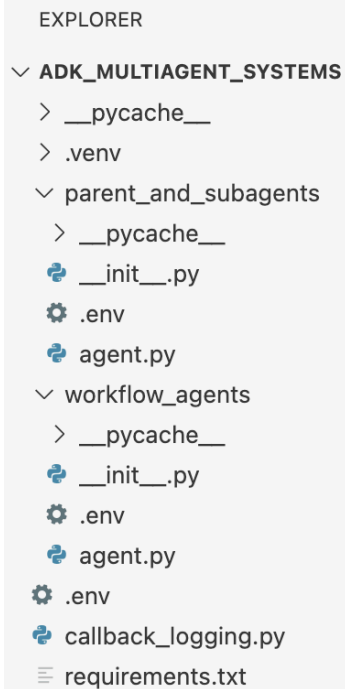
3. 將 `[YOUR-PROJECT-ID]` 替換為實際的 Google Cloud 專案 ID。(例如：`PROJECT_ID = "google-cloud-labs"`)
如果您不記得專案 ID，請在終端機中執行下列指令。畫面會列出所有專案及其 ID。

```
gcloud projects list
```

4. 在終端機中，將這個 `.env` 檔案複製到子代理程式目錄，讓子代理程式也能存取變數：

```
cp .env parent_and_subagents/.env
cp .env workflow_agents/.env
```

檔案結構現在應如下所示：



```
EXPLORER
└─ ADK_MULTIAGENT_SYSTEMS
  ├── __pycache__
  ├── .venv
  ├── parent_and_subagents
  │   ├── __pycache__
  │   ├── __init__.py
  │   ├── .env
  │   └── agent.py
  ├── workflow_agents
  │   ├── __pycache__
  │   ├── __init__.py
  │   ├── .env
  │   └── agent.py
  ├── .env
  ├── callback_logging.py
  └── requirements.txt
```

7. 瞭解父代理、子代理和同層代理間的轉移方向

對話一律會從 `root_agent` 開始。根據預設，父代理會使用子代理程式的 `description`，判斷何時應轉移對話。您也可以在此父代理的 `instruction` 中，使用子代理的 `name` 明確引導轉移方向。

我們來試試。

1. 在 Cloud Shell 編輯器中開啟 `adk_multiagent_systems/parent_and_subagents/agent.py`。請注意 `agent.py` 檔案中的三個代理：
 - `root_agent` (名為 `steering`)：向使用者提問，決定要轉移至哪個子代理。一開始，它只會依據子代理的 `description`。
 - `travel_brainstormer`：協助使用者發想目的地。
 - `attractions_planner`：協助使用者列出特定國家/地區的觀光景點。

提示：如要依對話流程順序閱讀 `agent.py` 檔案，最簡單的方法通常是從底部的 `root_agent` 開始往上閱讀。這是因為 Python 程式碼要求先定義子代理，才能新增至代理。

2. 在建立 `root_agent` 時加入下列程式碼，將 `travel_brainstormer` 和 `attractions_planner` 做為 `root_agent` 的子代理：

```
sub_agents=[travel_brainstormer, attractions_planner]
```

注意：代理程式階層只由父項的 `sub_agents` 清單定義，子代理不需要 `parent` 參數。

3. 在終端機中，與代理程式對話：

```
cd ~/adk_multiagent_systems
adk run parent_and_subagents
```

4. 在終端機的 `[user]:` 提示中，輸入：

```
hello
```

輸出內容範例 (可能與實際輸出內容略有不同)：

```
[steering]: Hi there! Do you already have a country in mind for your trip, or would
you like some help deciding where to go?
```

5. 現在，在終端機中告訴代理程式：

```
I could use some help deciding.
```

輸出內容範例 (可能與實際輸出內容略有不同)：

```
[travel_brainstormer]: Okay! To give you the best recommendations, I need to
understand what you're looking for in a trip.
...
```

請注意 `[travel_brainstormer]` 標記。 `root_agent` 轉移的控制權僅以子代理商的 `description` 為依據。

6. 在終端機的 `user:` 提示中，輸入 `exit` 並按下 Enter 鍵，即可結束對話。

7. 現在我們來明確一點。在 `agent.py` 中，將下列內容新增至 `root_agent` 的 `instruction`：

```
    If they need help deciding, send them to 'travel_brainstormer'.
    If they know what country they'd like to visit, send them to the
    'attractions_planner'.
```

8. 在終端機中再次執行代理：

```
adk run parent_and_subagents
```

9. 在終端機的 `[user]:` 提示中，輸入：

```
hello
```

10. 這時請回覆：

```
I would like to go to Japan.
```

輸出內容範例 (可能與實際輸出內容略有不同)：

```
[attractions_planner]: Okay, I can help you with that! Here are some popular
attractions in Japan:
...
```

按照新指示，將轉移作業通知傳送至 `attractions_planner`。

11. 現在請回覆：

```
Actually I don't know what country to visit.
```

輸出內容範例 (可能與實際輸出內容略有不同)：

```
[travel_brainstormer]: Okay! I can help you brainstorm some countries for travel...
```

你會發現對話已轉給 `travel_brainstormer`，也就是 `attractions_planner` 的同層代理。這項功能預設為允許。

如要禁止轉移，可以在 `attractions_planner` 代理上設定 `disallow_transfer_to_peers=True`。

12. 在使用者提示詞中輸入 `exit`，結束工作階段。

重點回顧

在本節中，您已瞭解代理程式階層和對話流程的基本概念：

- 對話一律會從 `root_agent` 開始。
 - 父代理可根據 `description` 自動轉移給子代理。
 - 您可以透過 `name`，明確控制將 `instruction` 轉移給子代理的流程。
 - 根據預設，代理可以轉移給 `peer` 代理 (階層中的同層代理)。
-

8. 使用工作階段狀態來儲存及擷取資訊

每段 ADK 對話都有 `Session`，其中包含工作階段狀態字典。所有代理都能存取這項狀態，因此非常適合在代理之間傳遞資訊，或在整個對話過程中維護資料 (例如清單)。

請按照下列步驟新增和讀取狀態：

1. 返回檔案 `adk_multiagent_systems/parent_and_subagents/agent.py`
2. 將下列函式定義貼到 `# Tools` 標頭後方：

```
def save_attractions_to_state(
    tool_context: ToolContext,
    attractions: List[str]
) -> dict[str, str]:
    """Saves the list of attractions to state["attractions"].

    Args:
        attractions [str]: a list of strings to add to the list of attractions

    Returns:
        None
    """
    # Load existing attractions from state. If none exist, start an empty list
    existing_attractions = tool_context.state.get("attractions", [])

    # Update the 'attractions' key with a combo of old and new lists.
    # When the tool is run, ADK will create an event and make
    # corresponding updates in the session's state.
    tool_context.state["attractions"] = existing_attractions + attractions

    # A best practice for tools is to return a status message in a return dict
    return {"status": "success"}
```

請注意，這段程式碼有以下幾點：

- 這個函式會接收 `tool_context: ToolContext`。這個物件是工作階段的閘道。
- `tool_context.state["attractions"] = ...` 行會直接從工作階段的狀態字典讀取及寫入資料。其餘工作交由 ADK 代勞。

3. 加入 `tools` 參數，將工具新增至 `attractions_planner` 代理：

```
tools=[save_attractions_to_state]
```

4. 在 `attractions_planner` 代理現有的 `instruction` 中，新增下列項目符號清單：

```
- When they reply, use your tool to save their selected attraction and then
provide more possible attractions.
- If they ask to view the list, provide a bulleted list of { attractions? } and
then suggest some more.
```

注意以大括號括住的區段：`{ attractions? }`。這項 ADK 功能 ([索引鍵範本](#)) 會將狀態中的 `attractions` 索引鍵值插入代理程式的提示。`?` 會將其設為選用，避免索引鍵尚不存在時發生錯誤。

5. 在終端機中執行下列指令，啟動 **Agent Development Kit** 網頁 UI：

```
adk web
```

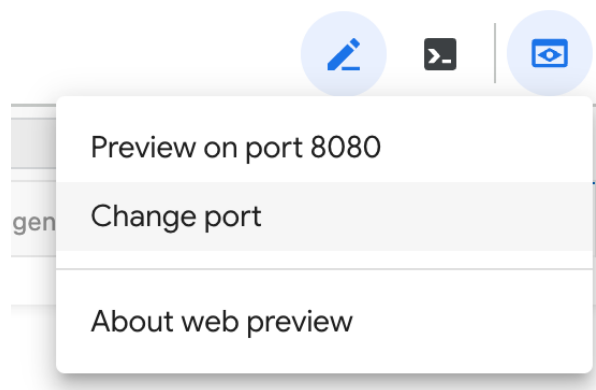
輸出內容

```
INFO:      Started server process [2434]
INFO:      Waiting for application startup.
+-----+
| ADK Web Server started |
|                         |
| For local testing, access at http://localhost:8000. |
+-----+

INFO:      Application startup complete.
INFO:      Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
```

注意：如果通訊埠 8000 正在使用中，您可以改用其他通訊埠執行，例如：`adk web --port 8001`。您必須在後續步驟中更新通訊埠編號，確保兩者一致。

6. 在 Cloud Shell 終端機中，按一下「Web Preview」(網頁預覽) 按鈕，然後選取「Change Port」(變更通訊埠)，即可在新分頁中查看網頁介面。



7. 輸入通訊埠編號 **8000**，然後按一下「變更並預覽」。系統會開啟新的瀏覽器分頁，顯示 ADK 開發 UI。

Change Preview Port

Port Number *

8000

[Cancel](#) [Change and Preview](#)

8. 在左側的「Select an agent」下拉式選單中，選擇 `parent_and_subagents`。

9. 輸入 `hello` 開始對話。

10. 代理向您打招呼後，請回覆：

I'd like to go to Egypt.

對話應會轉給 `attractions_planner`，並顯示景點清單。

11. 選擇景點，例如：

I'll go to the Sphinx

12. 您應該會收到類似以下的回覆：「好的，我已將人面獅身像存到清單中...

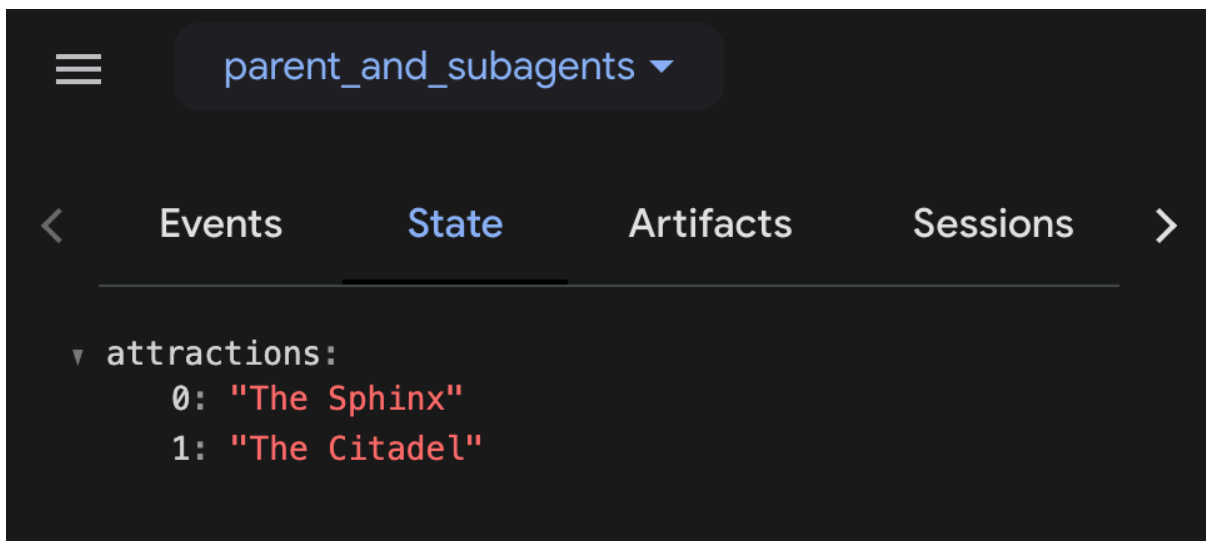
13. 點選標有勾號的回覆工具方塊，查看根據工具回覆建立的事件。

請注意，其中的「actions」欄位包含 `stateDelta`，作用是說明狀態變更。

14. 從服務專員的清單中回覆其他景點。

15. 在左側導覽選單點選「X」，退出先前查看的事件視窗。

16. 按一下左側邊欄的「狀態」分頁標籤。您現在可以在工作階段的狀態中看到 `attractions` 陣列，其中應包含您選取的兩個項目。



17. 向代理傳送下列訊息：

What is on my list?

代理程式現在應會從狀態讀取資料，並傳回清單。

18. 測試完代理後，請關閉網路瀏覽器分頁，接著在 Cloud Shell 終端機按下 **CTRL + C** 鍵來停止伺服器。

如要在狀態中儲存代理程式的完整文字回覆，您可以在定義代理程式時設定 `output_key` 參數，系統會將完整輸出內容儲存到該欄位名稱下的狀態字典。

章節回顧

在本節中，您學會如何使用 `Session` 狀態分享資料：

- **寫入狀態**：您可以使用 `tool_context.state` 物件 (例如 `tool_context.state["my_list"] = [...]`)，從工具內寫入狀態字典。
 - **讀取狀態**：使用鍵範本 (例如 `Here is your list: {my_list?}`)，將狀態資料直接插入代理程式的 `instruction`。
 - **檢查狀態**：您可以使用「State」分頁標籤，在 ADK 開發 UI 中即時監控工作階段狀態。
-

9. 工作流程代理

到目前為止，您已瞭解父代理如何轉移至子代理，然後等待使用者。工作流程代理則不同，這類代理會執行自動流程中的子代理，不必等待使用者輸入內容。

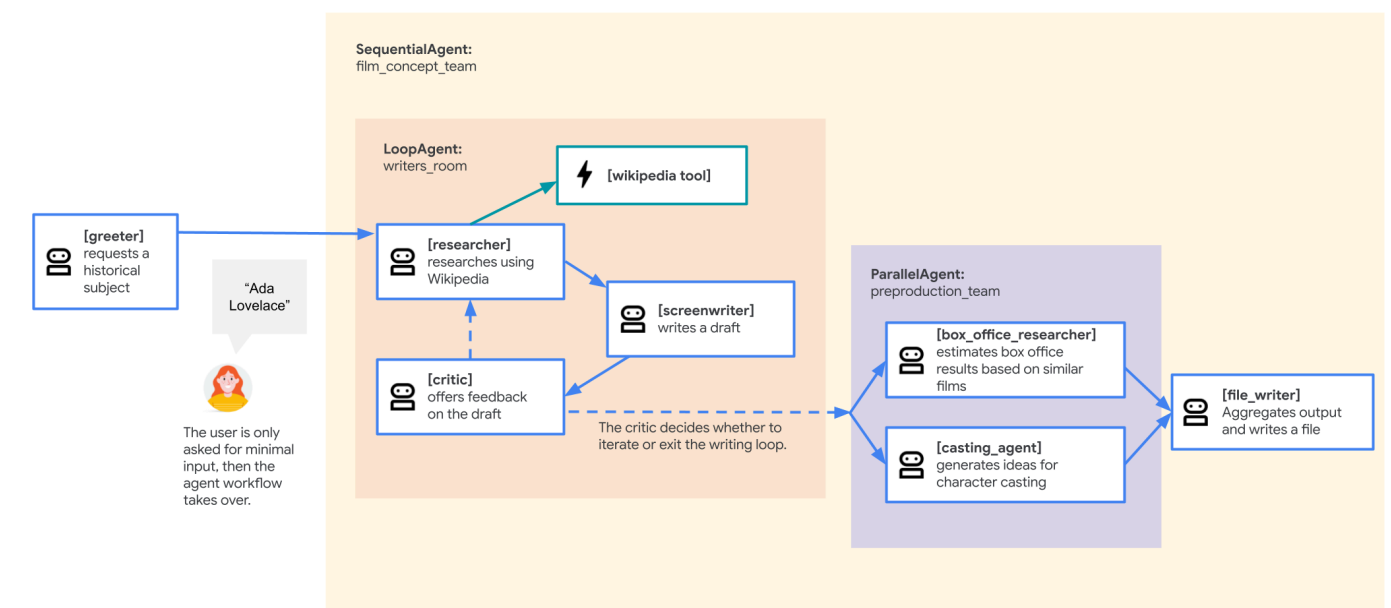
非常適合自動化多步驟工作，例如「規劃及執行」或「草擬及修訂」管道。ADK 提供三種內建工作流程代理，可管理這項作業：

- SequentialAgent
- LoopAgent
- ParallelAgent

本實驗室的後續部分將著重於使用這三個工作流程代理，建構多代理系統。

您需要建構代理，為新上映的歷史人物電影製作提案文件。代理會負責研究、反覆撰寫及生成報告。

最終系統會如下所示：

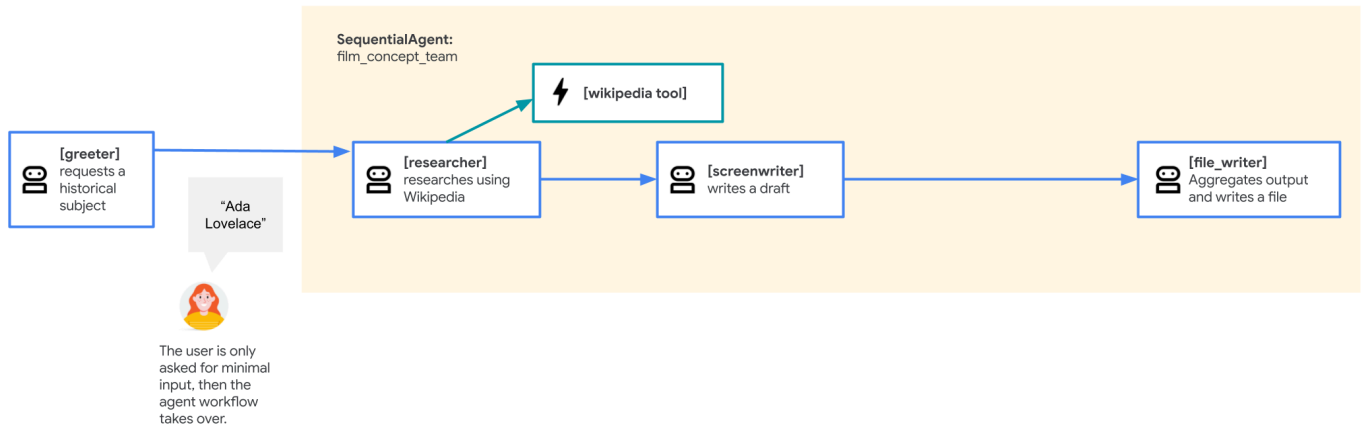


您將逐步建構這個系統，從最簡單的工作流程開始。

10. 使用 SequentialAgent 建構多代理系統

`SequentialAgent` 是指以簡單線性序列執行子代理的工作流程代理。也就是說，`sub_agents` 清單中的各個代理會依序運作。如果管道中的工作必須依特定順序執行，就非常適合使用 `SequentialAgent`，例如您現在要建構的電影提案代理程式。

第一個版本結構如下：



- `root_agent (greeter)` 會歡迎使用者，並取得電影主題。
- 然後轉移至名為 `film_concept_team` 的 `SequentialAgent`，該 `SequentialAgent` 會：
 1. 執行 `researcher` 代理程式，從維基百科取得事實。
 2. 執行 `screenwriter` 代理程式，根據這些事實撰寫劇情。
 3. 執行 `file_writer` 代理程式，將最終繪圖儲存至檔案。

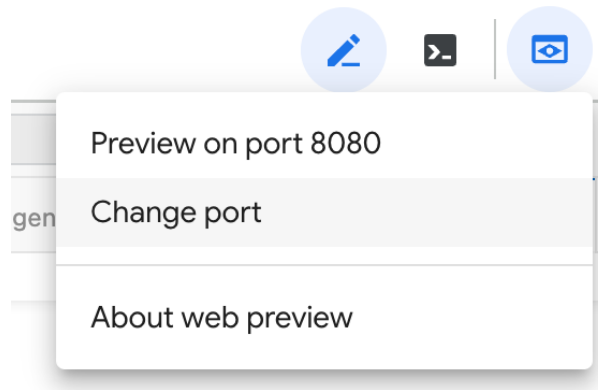
執行看看。

1. 在 **Cloud Shell 編輯器** 中開啟 `adk_multiagent_systems/workflow_agents/agent.py`。
詳閱這個代理的定義檔。由於必須先定義子代理，才能指派至父代理，如要依對話流程順序讀取檔案，請從檔案底部往上讀取代理。
2. 請注意 `append_to_state` 工具。代理程式可透過這個輔助函式，將資料附加至工作階段狀態中的清單，`researcher` 和 `screenwriter` 就是以這種方式傳遞工作。
3. 試用代理程式。在終端機中，啟動已啟用即時重新載入功能的網頁介面：

```
cd ~/adk_multiagent_systems
adk web --reload_agents
```

注意：如果通訊埠 8000 正在使用中，您可以改用其他通訊埠執行，例如：`adk web --reload_agents --port 8001`。

4. 在 Cloud Shell 終端機中，按一下「Web Preview」(網頁預覽) 按鈕，然後選取「Change Port」(變更通訊埠)，即可在新分頁中查看網頁介面。



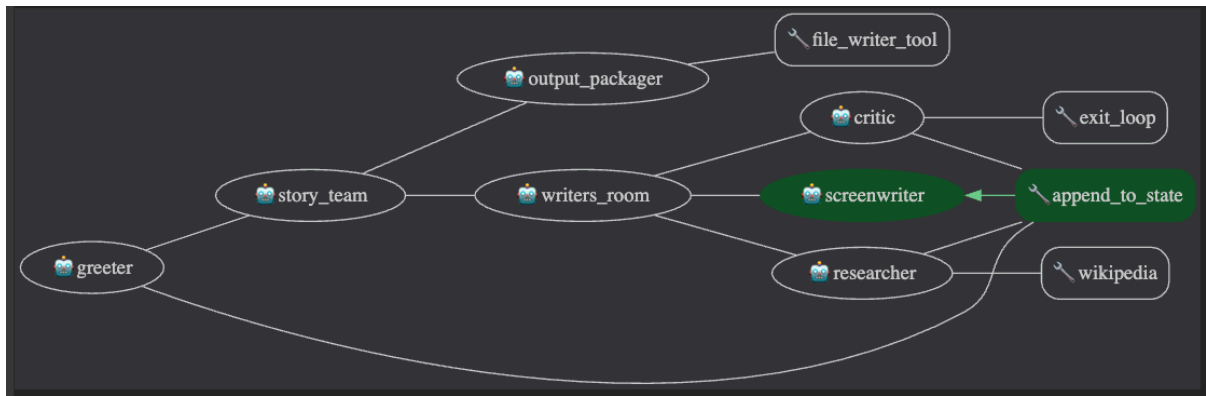
5. 輸入通訊埠編號 **8000**，然後按一下「變更並預覽」。系統會開啟新的瀏覽器分頁，顯示 ADK 開發 UI。

Change Preview Port

Port Number *

[Cancel](#) [Change and Preview](#)

6. 在「Select an agent」下拉式選單中，選取 `workflow_agents`。
7. 輸入 `hello` 開始對話。`greeter` 服務專員會回覆。
8. 代理提示您輸入歷史人物時，您可以使用這些範本，也可以自行製作：
- 張仲景
 - 林可欣
 - 馬可·奧理略
9. 現在由 `SequentialAgent` 接手。您不會看到任何中間訊息。`researcher`、`screenwriter` 和 `file_writer` 會依序執行。代理程式只會在整個序列完成時回覆。
- 如果失敗，請點選右上方的「+ New Session」，然後再試一次。
10. 代理程式確認檔案已寫入後，請在 Cloud Shell 編輯器的 `movie_pitches` 目錄中找出並開啟新的 `.txt` 檔案，查看輸出內容。
11. 在 ADK 開發人員使用者介面中，按一下對話記錄中的最後一個代理圖示，開啟事件檢視畫面。
12. 事件檢視畫面會顯示代理樹狀結構的視覺化圖表。您可以看到 `greeter` 如何呼叫 `film_concept_team`，然後依序呼叫每個子代理。



13. 您可以點選圖表中任何代理的「Request」和「Response」分頁標籤，檢查傳遞的確切資料，包括工作階段狀態。

注意：由於提示簡單，這個代理程式的輸出內容品質有限。在實際系統中，您會為每個代理提供更嚴謹的指令和範例，以提升可靠性。

章節回顧

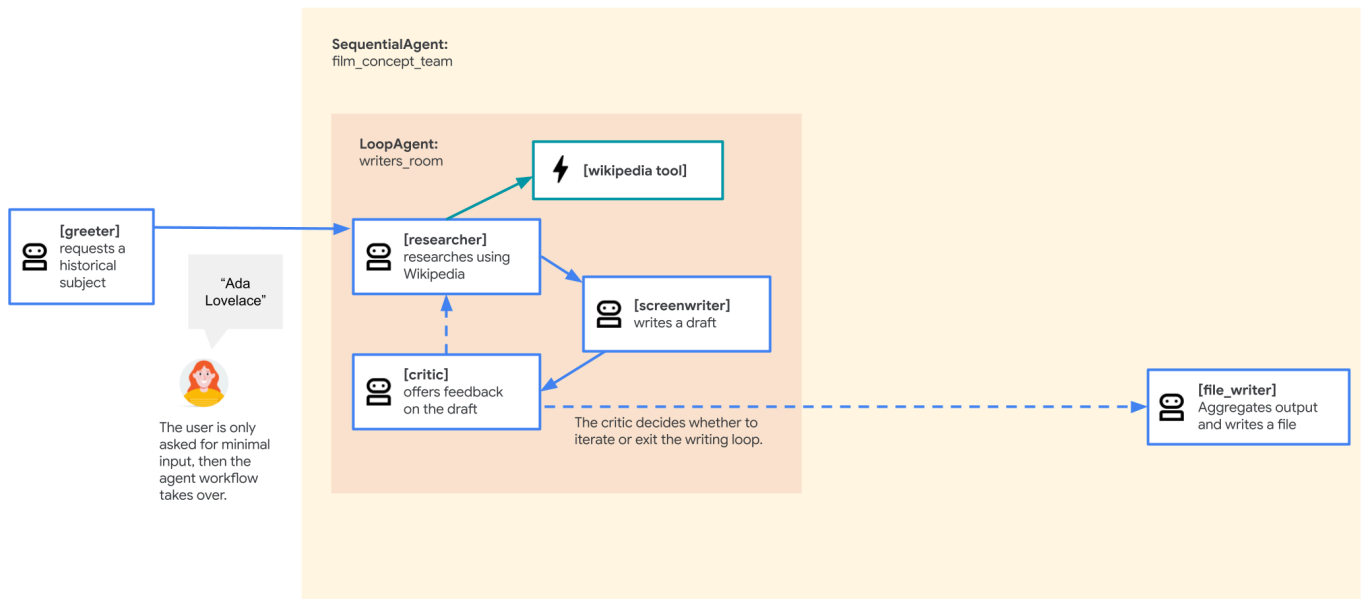
在本節中，您已瞭解如何使用**工作流程代理程式**：

- `SequentialAgent` 會依序逐一執行子代理，步驟之間不會等待使用者輸入。
- 這就是「工作流程」，因為使用者會與 `root_agent` 對話，然後將工作交給 `SequentialAgent` 完成。
- 序列中的子代理會使用**工作階段狀態** (例如 `{ PLOT_OUTLINE? }`) 存取先前代理的工作。
- 您可以使用開發 UI 中的**事件圖表**，以視覺化方式呈現並偵錯整個代理程式對代理程式的工作流程。

11. 新增 LoopAgent 來執行疊代作業

`LoopAgent` 是工作流程代理程式，會依序執行子代理程式，然後從頭開始重複執行。這個「迴圈」會持續執行，直到滿足條件為止，例如達到 `max_iterations` 計數，或子代理呼叫內建的 `exit_loop` 工具。

這項功能非常適合需要反覆修正的工作。您將新增這個 `LoopAgent`，為電影提案代理建立「編劇室」。這項功能可讓 `researcher`、`screenwriter` 和新的 `critic` 代理程式以迴圈方式運作，每次傳遞都會改善劇情，直到 `critic` 認為劇情已準備就緒。這也有助於代理程式處理較模糊的使用者輸入內容 (例如「古代醫生」)，讓代理程式研究並修正想法。



現在，請按照下列步驟進行變更：

1. 在 `adk_multiagent_systems/workflow_agents/agent.py` 中，加入 `exit_loop` 的匯入項目 (其他 `google.adk` 匯入項目附近)：

```
from google.adk.tools import exit_loop
```

2. 新增 **critic** 代理程式。這位代理人會審查劇情。如果良好，則會呼叫 `exit_loop`。如果不是，則會將意見回饋新增至下一個迴圈的狀態。

將下列代理定義貼到 `# Agents` 部分下方：

```
critic = Agent(
    name="critic",
    model=model_name,
    description="Reviews the outline so that it can be improved.",
    instruction=""
    INSTRUCTIONS:
    Consider these questions about the PLOT_OUTLINE:
    - Does it meet a satisfying three-act cinematic structure?
    - Do the characters' struggles seem engaging?
    - Does it feel grounded in a real time period in history?
    - Does it sufficiently incorporate historical details from the RESEARCH?

    If the PLOT_OUTLINE does a good job with these questions, exit the writing loop with
    your 'exit_loop' tool.

    If significant improvements can be made, use the 'append_to_state' tool to add your
    feedback to the field 'CRITICAL_FEEDBACK'.

    Explain your decision and briefly summarize the feedback you have provided.

    PLOT_OUTLINE:
    { PLOT_OUTLINE? }

    RESEARCH:
    { research? }
    "",
    before_model_callback=log_query_to_model,
    after_model_callback=log_model_response,
    tools=[append_to_state, exit_loop]
)
```

3. 建立 **writers_room** `LoopAgent`。其中包含會在迴圈中運作的三個代理程式。
將下列程式碼貼到 **film_concept_team** 代理定義「上方」：

```
writers_room = LoopAgent(
    name="writers_room",
    description="Iterates through research and writing to improve a movie plot outline.",
    sub_agents=[
        researcher,
        screenwriter,
        critic
    ],
    max_iterations=5,
)
```

請注意 `max_iterations=5` 參數。即使您有結束條件，這也是防止無限迴圈的重要安全措施。

4. 更新 **film_concept_team** `SequentialAgent`，改用新的 **writers_room** 迴圈。將 **researcher** 和 **screenwriter** 替換為單一 **writers_room** 代理程式。將現有的 **film_concept_team** 定義替換為以下內容：

```
film_concept_team = SequentialAgent(
    name="film_concept_team",
    description="Write a film plot outline and save it as a text file.",
    sub_agents=[
        writers_room,
        file_writer
    ],
)
```

5. 返回 ADK 開發 UI 分頁，然後點選右上方的「+ New Session」。

6. 輸入 `hello` 開始新的對話。

7. 系統顯示提示時，請提供更廣泛的主題。您可以參考以下構想：

- an industrial designer who made products for the masses
- 製圖師 (地圖製作者)
- that guy who made crops yield more food

代理程式現在會透過迴圈運作。在 ADK 開發人員使用者介面中，您會看到代理多次執行時的記錄 (例如「[researcher]」、「[screenwriter]」、「[critic]」、「[researcher]」、「[screenwriter]」、「[critic]...」)。

8. 迴圈完成後，代理程式就會寫入檔案。查看 `adk_multiagent_systems/movie_pitches` 目錄中產生的檔案。

9. 檢查開發人員使用者介面中的事件圖表，查看迴圈結構。

章節回顧

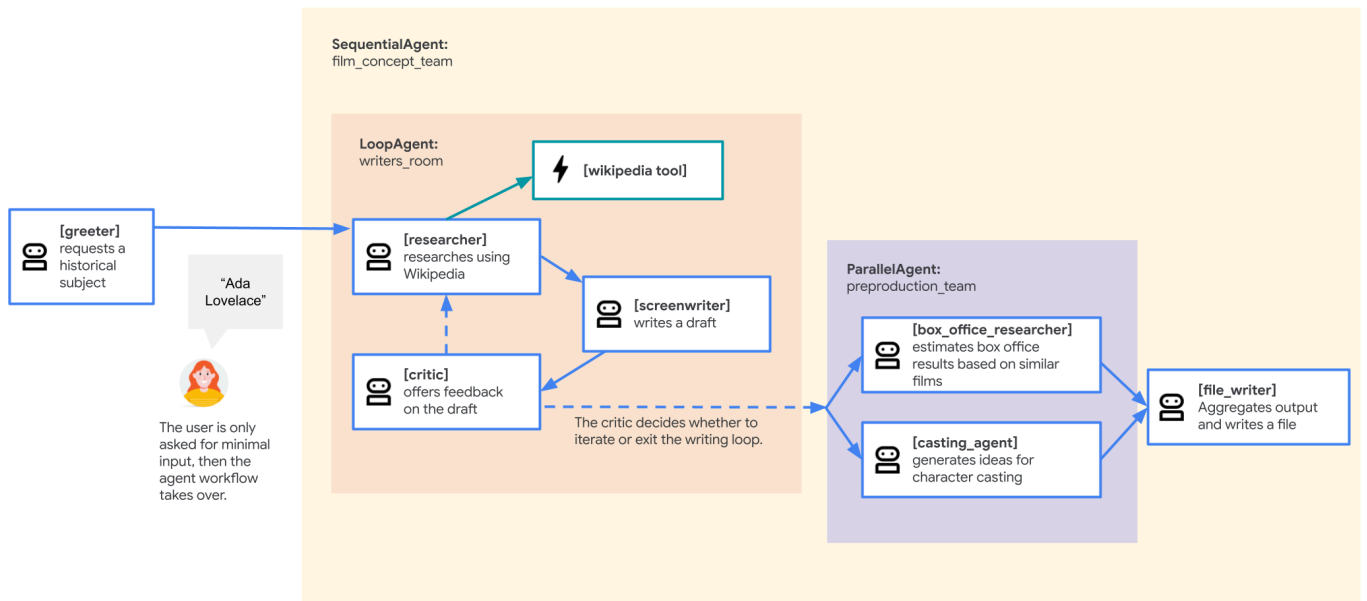
在本節中，您已瞭解如何使用 `LoopAgent`：

- `LoopAgent` 是工作流程代理程式，會重複執行子代理程式序列，為疊代工作建立「內部迴圈」。
 - 迴圈中的代理會在後續傳遞時，使用工作階段狀態相互傳遞工作 (例如 `PLOT_OUTLINE`) 和意見回饋 (例如 `CRITICAL_FEEDBACK`)。
 - 達到 `max_iterations` 限制或代理呼叫 `exit_loop` 工具時，迴圈就會停止。
-

12. 使用 ParallelAgent 執行「擴散收集」作業

`ParallelAgent` 是工作流程代理，會同時 (並行) 執行所有子代理。如果工作可劃分為獨立的子工作，例如執行兩項不同的研究工作，就很適合採用這種代理。

您將使用 `ParallelAgent` 建立「前置製作團隊」，並行作業。一位代理會研究票房潛力，另一位代理則同時腦力激盪選角想法。這通常稱為「分散與收集」模式：`ParallelAgent`「分散」工作，而後續的代理程式 (我們的 `file_writer`)「收集」結果。



最終的代理程式流程如下：

1. **greeter** (根) 會發起對話。
2. 並轉移至 `film_concept_team` (`SequentialAgent`)，該程序會執行：
 - 用於建立情節的 **writers_room** (`LoopAgent`)。
 - 新的 **preproduction_team** (`ParallelAgent`) 可同時研究票房和選角。
 - **file_writer**：收集所有結果並儲存檔案。

現在，請按照下列步驟進行變更：

1. 在 `adk_multiagent_systems/workflow_agents/agent.py` 中，將新的 `ParallelAgent` 和子代理貼到 `# Agents` 標頭下方。

```

box_office_researcher = Agent(
    name="box_office_researcher",
    model=model_name,
    description="Considers the box office potential of this film",
    instruction=""
    PLOT_OUTLINE:
    { PLOT_OUTLINE? }

    INSTRUCTIONS:
    Write a report on the box office potential of a movie like that described in
    PLOT_OUTLINE based on the reported box office performance of other recent films.
    "",
    output_key="box_office_report"
)

casting_agent = Agent(
    name="casting_agent",
    model=model_name,
    description="Generates casting ideas for this film",
    instruction=""
    PLOT_OUTLINE:
    { PLOT_OUTLINE? }

    INSTRUCTIONS:
    Generate ideas for casting for the characters described in PLOT_OUTLINE
    by suggesting actors who have received positive feedback from critics and/or
    fans when they have played similar roles.
    "",
    output_key="casting_report"
)

preproduction_team = ParallelAgent(
    name="preproduction_team",
    sub_agents=[
        box_office_researcher,
        casting_agent
    ]
)

```

2. 更新 `film_concept_team` `SequentialAgent` 的 `sub_agents` 清單，加入新的 `preproduction_team` (介於 `writers_room` 和 `file_writer` 之間)。將現有的 `film_concept_team` 定義替換為以下內容：

```

film_concept_team = SequentialAgent(
    name="film_concept_team",
    description="Write a film plot outline and save it as a text file.",
    sub_agents=[
        writers_room,
        preproduction_team,
        file_writer
    ],
)

```

- 更新 `file_writer` 代理程式的 `instruction`，以便「收集」狀態中的新報表，並將其新增至檔案。
將 `file_writer` 的 `instruction` 字串替換為：

```

instruction="""
INSTRUCTIONS:
- Create a marketable, contemporary movie title suggestion for the movie described in
the PLOT_OUTLINE.
If a title has been suggested in PLOT_OUTLINE, you can use it, or replace it with a
better one.
- Use your 'write_file' tool to create a new txt file with the following arguments:
- for a filename, use the movie title
- Write to the 'movie_pitches' directory.
- For the 'content' to write, include:
- The PLOT_OUTLINE
- The BOX_OFFICE_REPORT
- The CASTING_REPORT

PLOT_OUTLINE:
{ PLOT_OUTLINE? }

BOX_OFFICE_REPORT:
{ box_office_report? }

CASTING_REPORT:
{ casting_report? }
""",

```

- 返回 ADK 開發人員使用者介面分頁，然後按一下「+ New Session」。
- 輸入 `hello` 即可開始對話。
- 代理提示時，輸入新的人物構想，您可以參考以下構想：
 - 發明 Wi-Fi 技術的女演員
 - an exciting chef
 - 世界博覽會展覽的關鍵參與者
- 代理完成工作後，請前往 `adk_multiagent_systems/movie_pitches` 目錄查看最終檔案。現在，這份文件應該會包含劇情、票房報告和演員報告。

章節回顧

在本節中，您已瞭解如何使用 `ParallelAgent`：

- `ParallelAgent` 「分散」工作，同時執行所有子代理，而非依序執行。
 - 對於互不相依的工作 (例如研究兩個不同的主題)，這項功能非常有效率。
 - 後續代理程式會「收集」平行代理程式的結果。做法是讓平行代理將工作儲存至工作階段狀態 (使用 `output_key`)，並讓最終代理 (例如 `file_writer`) 讀取這些鍵。
-

13. 自訂工作流程代理

如果預先定義的工作流程代理 `SequentialAgent`、`LoopAgent` 和 `ParallelAgent` 無法滿足需求，不妨使用 `CustomAgent` 彈性導入新的工作流程邏輯。

您可以定義子代理間的流程控制、條件式執行或狀態管理模式。這個代理適用於複雜的工作流程、有狀態的自動化調度管理作業，或將自訂商業邏輯整合至框架的自動化調度管理層。

本實驗室不會說明如何建立 `CustomAgent`，但您可以先認識這個代理，需要時就能使用！

14. 恭喜！

您已使用 [Google Agent Development Kit \(ADK\)](#) 成功建構精密的[多代理系統](#)。您已從簡單的父項-子項代理關係，進展到自動化調度複雜的工作流程，可研究、撰寫及潤飾廣告素材專案。

重點回顧

在本實驗室中，您完成了下列工作：

- 以階層式樹狀結構整理代理程式，並建立父項和子代理關係。
- 控管代理程式對代理程式的轉移，包括自動 (使用 `description`) 和明確 (使用 `instruction`) 轉移。
- 使用工具將資料寫入 `tool_context.state` 字典。
- 使用鍵範本 (例如 `{ PLOT_OUTLINE? }`) 從工作階段狀態讀取資料，並引導代理的提示。
- 實作 `SequentialAgent`，建立簡單的逐步工作流程 (研究 -> 撰寫 -> 儲存)。
- 使用 `LoopAgent` 和 `critic` 代理程式，以及 `exit_loop` 工具，建立反覆修正週期。
- 使用 `ParallelAgent` 「分散」獨立工作 (例如選角和票房研究)，以便同時執行。

持續進行實驗

你可以透過多種方式，進一步運用所學知識。不妨參考下列建議：

- **新增更多代理：**嘗試在 `preproduction_team` `ParallelAgent` 中新增代理。舉例來說，您可以建立 `marketing_agent`，根據 `PLOT_OUTLINE` 為電影撰寫宣傳標語。
 - **新增更多工具：**為 `researcher` 代理新增更多工具。您可以建立工具，使用 Google 搜尋 API 尋找 Wikipedia 上沒有的資訊。
 - **探索 `CustomAgent`：**實驗室提到不符合標準範本的工作流程 `CustomAgent`。舉例來說，您可以嘗試建構一個代理程式，只有在工作階段狀態中存在特定鍵時，才會有條件地執行。
-